```
  external_labels:
    env: "docker"
rule_files:
- alert.rules
alerting:
  alertmanagers:
  - static_configs:
    - targets:
      - "alertmanager:9093"
scrape_configs:
  - job_name: "prometheus"
    static_configs:
    - targets: ["localhost:9090"]
```

References where the alerting rule is defined

Configures the Alertmanager

Defines what to scrape (in this case, Prometheus self-scrape)

Our alert is defined in alert.rules, shown in the following listing. We want to fire the alert once the total number of HTTP requests Prometheus sees exceeds a certain number (40, in our example).

Listing 7.3 Alerting rule

```
groups:
- name: alert.rules
  rules:
  - alert: TooManyAPICalls
    expr: sum(prometheus_http_requests_total) > 40
    labels:
      severity: "critical"
    annotations:
      summary: "Too many API calls"
      description: "The Prometheus instance {{ $labels.instance }} is
       experiencing too
       many API calls"
```

The name of our alerting rule

The PromQL expression Prometheus evaluates

Using a Prometheus label to communicate how important the alert is

The Alertmanager config alertmanager.yml is depicted next.

Listing 7.4 Alertmanager configuration

```
route:
  group_by: ["env"]
  receiver: "mywebhook"
receivers:
- name: "mywebhook"
  webhook_configs:
  - url: "http://webhook:80/anything"
```

The grouping definition. We want all alerts with the env label grouped together.

Defining the receiver (how to send the notification) for this route

Configures the receiver. In this case, we want the notification via a webhook.

If you go to http://localhost:9090, you should see the Prometheus UI. You can find the alerting rules and their status, as shown in figure 7.3, via the top-level Alerts menu item. If you want to speed up things, you need to perform a few queries (e.g., using the PromQL query `sum(prometheus_http_requests_total)`) or, otherwise, wait a few minutes until you see the alert firing.